

LDPC-Staircase Codes for Soft Decision Decoding

V. B. Wijekoon, Emanuele Viterbo, Yi Hong
Monash University, Australia

Abstract—Staircase codes, a class of product-like codes, have been demonstrated to perform exceptionally well in optical transmission systems. Although they are predominantly used with BCH component codes and hard decision decoding, soft decision decoding has also been recently attempted, with BCH and polar code based staircase codes. We consider using LDPC codes as the component code of soft decoded staircase codes. Results demonstrate that these codes offer very good performance, with gains in the range of 0.5-1dB over soft decoded BCH-staircase codes, at a BER of 10^{-8} . These can be further improved through the novel bit-flipping scheme we propose.

I. INTRODUCTION

Staircase codes, introduced in [1], is a class of codes which has characteristics of both block and convolutional codes. These codes are structured similar to product codes, and can be thought of as ongoing chains of product code blocks. Initially, staircase codes were constructed with very high rate BCH codes as the component codes, and the resulting high rate codes were targeted for optical transmission systems. In such a setting, and with hard decision decoding (HDD), they were shown to offer impressive output bit error-rates of 10^{-15} at a gap of only 0.56dB from capacity [1].

Sliding window decoding is usually used for staircase codes, in order to keep the decoding latency to a minimum [1]. For each window, decoding would be done iteratively. Although this scheme is advantageous in terms of latency and complexity, it is susceptible to undesirable error-floors, due to certain unresolvable error patterns. These are popularly known as ‘stall patterns’, and several ‘bit-flipping’ modifications have been suggested for sliding window decoding in order to resolve such patterns [2], [3].

Since optical communication systems are most often used with *streamed* data, decoding latency becomes a critical performance metric. In such a scenario, staircase codes, used with sliding window decoding, is quite an attractive option [1],[2]. Other than the obvious advantage of a sliding window structure of these codes offer further latency gains, which will be elaborated in Section II.

Although originally considered as unterminated chains of code blocks, proper termination of staircase codes has attracted some interest recently. Directly terminating the code is not a good option as this leads to the bits/symbols in the last block being protected only once, since they are part of a single step of the staircase, and as such, those bits become more error-prone. A popular method of termination is to add a few all-zero message blocks to the end, accepting a slight loss in rate [3], [4]. Alternatively, tail-biting schemes have also been studied in the literature [5].

Excellent performance reported with staircase codes are through HDD [1]. Obtaining further gains in performance by

incorporating some soft information in to the decoding setup of these codes has been attempted recently. One approach in this direction is to use staircase codes (with HDD) as outer codes in concatenated schemes, with soft decodable inner codes, such as low density parity-check (LDPC) [6] or low density generator matrix (LDGM) codes [7]. In [8], the authors consider marking received bits as highly reliable or unreliable, based on channel information, as a form of protection against mis-corrections. Gains of around 0.2 – 0.3dB over pure HDD are reported for BCH-staircase codes. Using soft decision decoding (SDD) directly on staircase code is an alternative approach, which requires soft decodable component codes. Such a setup for original BCH code based staircase codes is described in [4], where the well known Chase-Pyndiah algorithm [9] is used for SDD. Significant gains over HDD is reported, over AWGN channel, although this gain seems to diminish with component code length. Soft decodable polar-staircase codes have also been studied [10].

First proposed by Gallager in 1962 [11], LDPC codes came to prominence in the 1990s, and since then have been used in many communication standards, including those of Ethernet, digital television, and Wi-Fi. Capacity approaching performance under SDD and relatively low complex, iterative decoding algorithms are the major reasons for the popularity of these codes [12]. Due to these factors, we propose using LDPC codes as the component codes of soft decoded staircase codes. We introduce certain modifications to the sliding window decoding scheme for LDPC-staircase codes, and demonstrate their excellent performance through simulation results. As a means of lowering the error-floor, a novel bit-flipping scheme is proposed, which has minimal impact on decoding complexity.

Rest of the paper is organized as follows. Section II introduces LDPC-staircase codes, and discusses their encoding, termination, and decoding. Section III presents the novel bit-flipping scheme, along with a complexity analysis, while section IV presents the simulation results. Section V concludes the paper.

II. LDPC-STAIRCASE CODES

A. Encoding

For constructing an LDPC-staircase code, we first choose a $(2m, 2m - r)$ systematic LDPC code with parity bits at the end, to be used as the component code. The staircase code can be thought of as a sequence $B_0, B_1, B_2, \dots$ of $m \times m$ blocks. Block B_0 is initialized to a reference state known at both the transmitter and the receiver, such as the all-zero state, and is not transmitted. Encoding of each block $B_i (i \neq 0)$ is then carried out in a recursive fashion as follows.

In each block B_i , the leftmost $(m - r)$ columns are first filled with information symbols, and we will denote that sub-

matrix with $B_{i,S}$. Then, the encoder forms the $m \times (2m - r)$ matrix $A = [B_{i-1}^T \ B_{i,S}]$, where B^T represents the transpose of matrix B , and encodes each row with the $(2m, 2m - r)$ component LDPC code. Parity bits generated are added to the remaining r columns of block B_i . This process may be visualized as the blocks being arranged in a staircase fashion, as depicted in Fig. 1.

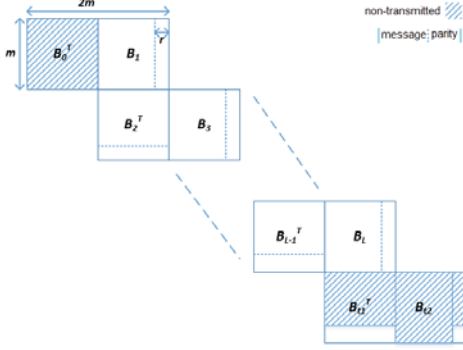


Fig. 1. A Terminated Staircase Code Block with L Information Blocks, and 2 Termination Blocks

B. Termination

In applications with a continuously streaming information source, such as in optical transmission networks, staircase codes may be used without termination [1]. Code rate in this case, with a $(2m, 2m - r)$ component code, is given by (1)

$$R_u = 1 - \frac{r}{m} \quad (1)$$

But in many other practical scenarios where staircase codes might be used advantageously, for example, in flash memories [3], they have to be properly terminated. Directly terminating the staircase code results in the final information block being protected *only once*, making those bits less reliable. This leads to error performance being dominated by the higher number of errors in the last block [3]. As extra protection for the final information block, an all-zero information block can be added to the staircase code [3], [4]. Additional $m \times r$ parity bits that would then be generated in the encoding process, which are not guaranteed to be zero, still have to be transmitted, resulting in a slight loss of rate. In Fig.1, B_L represents the last information block, while B_{t_1} represents the first all-zero information block.

Now the parity bits in B_{t_1} are only protected once, and this is cause for concern, as the errors in those bits could propagate into B_L , and then into B_{L-1} and so on, corrupting the decoding process. As a remedy, we add another all-zero information block, B_{t_2} . In the encoding process, first $(m - r)$ rows of $A = [B_{t_1}^T \ B_{t_2,S}]$ correspond to all-zero messages, and the parities generated for those rows are guaranteed to be zero. Therefore, in this case, only $r \times r$ parity bits have to be transmitted, and the loss of rate is almost negligible for sufficiently large L . It is possible to keep on adding all-zero message blocks, for each of which only $r \times r$ bits have to be transmitted, in the hope of improving performance. With L information blocks, and T additional *termination* blocks, final

code rate is given by (2).

$$R_t = \frac{m(m - r)L}{m^2L + mr + r^2(T - 1)} \quad (2)$$

It was observed, through simulations, that adding more than two additional blocks do not improve performance significantly. Therefore, in this paper, we focus on a setup with L information blocks and 2 termination blocks ($T = 2$ in (2)), as depicted in Fig.1.

Structure of LDPC-staircase codes is reminiscent of another very popular class of codes, the spatially-coupled LDPC (SC-LDPC) codes [13]. In fact, LDPC-staircase codes can be interpreted as a special case of SC-LDPC codes, where each codeword in a matrix $A_i = [B_{i-1}^T \ B_i]$ is ‘coupled’ with all the codewords in the two adjacent matrices A_{i-1} and A_{i+1} . This special form of coupling speeds up the decoding process, as would be evident from the next sub-section.

C. Decoding

Sliding window decoding scheme originally proposed in [1] is used to decode LDPC-staircase codes as well, with a few important modifications. General decoding procedure is outlined in the following, with these modifications highlighted.

Consider sliding window decoding with a window size w . For the i -th decoding operation, decoder utilizes blocks $B_i, B_{i+1}, \dots, B_{i+w-1}$, and the output is the final estimate for B_i ($i \geq 1$). Decoding is done iteratively for v_{max} iterations at most, or till no errors can be detected inside the window. For $(i + 1)$ -th decoding operation, decoder then *slides over* by one block. Since the codes we propose are terminated, if $w > 3$, window size would gradually decrease at the very end of the sequence, where there might not be w blocks available. For example, in the L -th decoding operation, window would only consist of 3 blocks; B_L, B_{t_1}, B_{t_2} .

Each decoding iteration starts from the first block in the window, B_i , and proceeds to the last block, B_{i+w-1} . This order is followed in the hope of propagating more reliable information through the decoder, since decoding of B_i is done using B_{i-1} , which has been decoded previously, or, in the case of $i = 1$, pre-known, and thus, contains more reliable information. Block B_j is decoded by individually decoding the rows of matrix $A = [B_{j-1}^T \ B_j]$, which are component codewords.

In SDD, two kinds of information are available for the decoder, *intrinsic* information from the channel and *extrinsic* information generated through calculations based on the structure of the code. These two types of information need to be handled in different ways, especially as extrinsic information gets continually updated in the decoding process. We use matrices $L_i^I, \dots, L_{i+w-1}^I$ for storing intrinsic information, and $L_i^E, \dots, L_{i+w-1}^E$ for extrinsic information.

In LDPC-staircase codes, extrinsic information is generated by sum-product algorithm (SPA) [12], the component code decoder. In particular, we use log-likelihood ratio (LLR) domain implementation of SPA. Interestingly, since SPA is also an iterative algorithm, this makes the whole decoding process *doubly iterative*, with I_{max} set as the maximum number of iterations for SPA.

Consider decoding rows of the matrix $A = [B_{j-1}^T \ B_j]$ (or decoding B_j). What decoder actually evaluates is a matrix of LLR values, which we denote with $L^c = [L_{j-1}^T \ L_j]$, where $L_{j-1} = L_{j-1}^I + L_{j-1}^E$ and $L_j = L_j^I + L_j^E$. Output of decoding each row is an updated vector of LLRs, from which the extrinsic part is extracted by subtracting from it the input LLR vector [9]. Those values are then used to *replace* the corresponding entries of L_{j-1}^E and L_j^E . Here, similar to [9], we propose using scaling factors to reduce propagation of errors as follows.

Extrinsic information in the instances where the inner decoder converges can be thought of as more reliable than where it does not. Thus, two separate scaling factors are used for the two cases; α_c for the first, and α_n for the second. Extrinsic information update after decoding the k -th row of A is carried out in two steps as follows. Here, ϵ denotes the extrinsic information vector obtained from soft decoding.

$$L_{j-1}^E(:, k) = \alpha_c \epsilon(1 : m) \quad (3)$$

$$L_j^E(k, :) = \alpha_n \epsilon(m + 1 : 2m) \quad (4)$$

Here, $\alpha = \alpha_c / \alpha_n$, depending on convergence of SPA.

A frame of an LDPC-staircase code contains a number of bits that are not transmitted, such as block B_0 , and the majority of blocks B_{t_1} and B_{t_2} . As we set these bits to zero, and use traditional BPSK transmission, the corresponding LLR values are set to high magnitude positive values in decoding. Also, it was observed in simulations that the first block of the sequence (B_1) is more susceptible to errors. Although it benefits from the high reliability of un-transmitted block B_0 , B_1 is part of only the first decoding window, and its rows undergo a lesser number of SPA iterations compared with those of other blocks. Therefore, we propose using a higher number of sliding window iterations ($v_{I,max}$) for the very first decoding operation.

Windowed decoding is also the decoding approach of choice for SC-LDPC codes [13]. Still, operations inside a decoding window of an LDPC-staircase code and a general SC-LDPC code are significantly different. SC-LDPC code carries out BP over the entire bi-partite graph inside the window, while in LDPC-staircase codes, decoding is more ‘localized’, as it considers the individual codewords separately from each other. Thus, a single BP iteration of an SC-LDPC code is comparable with one outer iteration, which constitutes of decoding a number of component codewords, of an LDPC-staircase code. This makes it appear as if LDPC-staircase codes are at a complexity disadvantage, since many component codewords have to be decoded inside a window, but it should be noted that component codes are usually short length LDPC codes, while SC-LDPC codes are generally constructed by coupling long LDPC codes. Also, number of outer iterations in decoding an LDPC-staircase code would be much lower than the number of BP iterations in windowed decoding of an SC-LDPC code. Thus, for the same window size in bits, windowed decoding of LDPC-staircase codes would be significantly faster than that of a general SC-LDPC code. While the *localized* decoding approach fails to take into account the constraints imposed from the coupling in BP decoding of a component codeword, it has a highly parallelizable structure, which would be beneficial in terms of latency.

III. BIT-FLIPPING SCHEME

In hard-decoded staircase codes, certain unresolvable error patterns exist, which are known as ‘*stall patterns*’ in the literature [1]. Error-floors of staircase codes are primarily due to these stall patterns, and upper bounds for error-floor have been calculated based on them [1], [2]. Although stall patterns cannot be resolved through traditional sliding window decoding, certain modifications have been introduced that do manage to resolve the majority [2], [3].

From hereon, we adopt a terminology similar to the one used in [1]. When we say ‘*syndromes of block B_j* ’, we mean the syndromes of the codewords that *end* in B_j , specifically syndromes of the rows of matrix $A = [B_{j-1}^T \ B_j]$.

A non-zero syndrome (of any block) inside the decoding window signals an existing error. If we disregard the cases where a row is decoded to the wrong codeword, then the *possible locations* of errors are the intersection points of non-zero syndrome rows/columns. In [2], [3], stall patterns are resolved by flipping all/some of the bits at these points, and optionally carrying out a few more sliding window iterations.

In order to further improve performance of LDPC-staircase codes, we use this insight on possible locations of errors, and devise a novel bit-flipping scheme. This scheme is outlined in Algorithm 1. Note that bit-flipping is attempted after each sliding window decoding operation, which was explained in Section II-C.

Algorithm 1 Sliding Window Decoder with Bit-flipping

Input: Blocks $B_0, \dots, B_{t_2}$ of staircase code

Output: Estimates for $B_1, \dots, B_L$

```

1: for  $i = 1 : L$  do
2:   // Regular sliding window decoding
3:
4:   // Bit-flipping Scheme
5:    $F \leftarrow \text{true}$ 
6:   if  $F$  then
7:      $Z_i \leftarrow$  non-zero syndrome rows of  $B_i$ 
8:      $Z_{i+1} \leftarrow$  non-zero syndrome rows of  $B_{i+1}$ 
9:     if  $0 < |Z_i| |Z_{i+1}|^2 < t_{flip}$  then
10:      // for each non-zero synd. row of  $B_i$ 
11:      for  $r_j \in Z_i$  do
12:         $S \leftarrow \text{false}$ 
13:         $q_j \leftarrow \emptyset$ 
14:         $z_{i,j} \leftarrow$  syndrome value
15:         $p_j \leftarrow$  intersection points
16:         $(S, q_j) \leftarrow \gamma_1(r_j, z_{i,j}, p_j)$ 
17:        if  $!S$  then
18:           $(S, q_j) \leftarrow \gamma_2(r_j, z_{i,j}, p_j)$ 
19:        if  $S$  then
20:          // Change LLR values of
21:          // positions in  $q_j$ 
22:        else
23:           $F \leftarrow \text{false}$ 
24:      else
25:         $F \leftarrow \text{false}$ 
26:    else if  $|Z_i| = 0$  then
27:       $F \leftarrow \text{true}$ 

```

Our focus is on resolving the errors of the *output block*,

B_i , as much as possible. A non-zero syndrome row in B_i could be due to errors in that block, or due to errors in B_{i-1} , the previous output block. In the second case, we cannot guarantee that the non-zero syndromes are only due to errors at intersections of such rows/columns in current decoding window. Indeed, due to errors in B_{i-1} , B_i could have non-zero syndrome rows but still have no errors. Thus, flipping scheme is only ‘activated’ if B_{i-1} had zero syndromes for all rows. State variable F (line 5 in Alg. 1) keeps track of this, and is set to *true* when decoding starts (since B_0 has no errors). Whenever an error is unresolvable, F is set to *false* (line 17-20).

Bit-flipping starts with calculating syndromes of blocks B_i and B_{i+1} (line 6-7). Indices of non-zero syndrome rows/columns of the two blocks are recorded in sets Z_i and Z_{i+1} , respectively. Additionally, Z_i stores the non-zero syndromes themselves. In line 8, a further check is done on the sizes of Z_i and Z_{i+1} (with the pre-set limit t_{flip}), in order for flipping to proceed. Reasons for this condition, which are related to complexity, would be elaborated on later in this section.

Two types of flipping would be considered for each non-zero syndrome row/column of B_i . First, we try flipping each of the intersection points, and if this does not resolve the errors of that row, we try with pairs of such points (line 12-14). These two types of operations are represented with the functions $\gamma_1(r_j, z_{i,j}, p_j)$ and $\gamma_2(r_j, z_{i,j}, p_j)$ in Alg. 1. Each of these functions require the hard decision of the non-zero syndrome row/column (r_j), its syndrome value ($z_{i,j}$), and the sets of intersection points (p_j). They output a Boolean variable S , signaling whether the bit-flipping operation was successful or not, and q_j , the set of indices of flipped bits. Inside the functions γ_1 and γ_2 , syndromes are re-calculated (using $z_{i,j}$ ’s) to check whether the errors have been resolved. If flipping a certain bit/a pair results in an all-zero syndrome, LLR values (in matrices L_i^I and L_i^E , in Section II-C) are changed accordingly (line 15-16). Note that flipping pairs of bits is not considered if some single bit flip results in an all-zero syndrome (line 14).

Calculating syndromes multiple times might seem a big cost on complexity, but through *updating* the originally calculated syndrome (stored in Z_i), this can be significantly reduced. Consider flipping the k -th bit of a vector. Only changes in the syndrome would be at the positions which correspond to parity-check equations containing that bit. New value of such a position can be simply calculated by adding 1 to the original value. This costs only a single XOR operation. Thus, updating the syndrome for a single bit flip would cost d_v XOR operations, where d_v denotes the number of equations (rows) in the parity-check matrix in which the flipped bit participates, which is known as the *variable node degree* in LDPC code terminology. Assuming a *regular* LDPC code, worst case complexity of bit-flipping, in the number of XOR operations required, can be calculated as follows.

$$\begin{aligned} \text{Cost for single bit-flips} &= |Z_i||Z_{i+1}|d_v \\ \text{Cost for flipping pairs of bits} &= 2|Z_i|\binom{|Z_{i+1}|}{2}d_v \\ \text{Total Cost} &= |Z_i||Z_{i+1}|d_v + 2|Z_i|\binom{|Z_{i+1}|}{2}d_v = |Z_i||Z_{i+1}|^2d_v \end{aligned}$$

As can be seen, complexity depends directly on the quantity

$|Z_i||Z_{i+1}|^2$. This is the reason for the limit enforced in line 8 of Algorithm 1. By using an appropriate value for t_{flip} , it is ensured that bit-flipping is not attempted in cases where the worst case complexity cost is too high. Although value d_v also affects complexity, its impact is disregarded since, with LDPC codes, d_v is usually a very small value (eg: 3,4). Also, it should be noted that in Alg. 1, bit-flipping (with a particular non-zero syndrome row) terminates as soon as an all-zero syndrome is reached. Worst case corresponds to instances where, for every non-zero syndrome row of B_i , all 1-bit and 2-bit flips result in non-zero syndromes, which is quite rare. Thus, in most cases, complexity of bit-flipping would be a fraction of the worst case cost $|Z_i||Z_{i+1}|^2d_v$.

It seems probable that even more errors may potentially be corrected if we consider additional ‘flipping patterns’, such as flipping sets of 3 bits, and so on. Obvious reason for not considering those is the increase in complexity, but there is a second reason as well. We do not want to *cause* extra errors through flipping, which may propagate through the sequence of blocks. Such errors can be caused by choosing the incorrect codeword for some rows. Since we are considering only 1-bit or 2-bit flips, this is only possible if the iterative decoder has already converged quite close to the wrong codeword. It has been observed that converging to a wrong codeword is rare for LDPC codes with reasonable minimum distances [14]. Thus, if a component code with good d_{min} is selected, and only 1-bit or 2-bit flips are considered, probability of introducing more errors through bit-flipping will be very small.

IV. SIMULATION RESULTS

In this section, we evaluate performance of the LDPC-staircase coding scheme, with all the modifications introduced in the previous sections.

First up, we consider high rate (around 0.85) staircase codes, such as the code based on the shortened (504, 468) BCH code, hereafter referred to as C_1 , used in [4]. Performance of two LDPC-staircase codes are compared against that of C_1 ; a code based on the (464, 434) LDPC code obtained from [15] (C_2), and another based on the (1440, 1344) LDPC code specified in IEEE 802.15.3c standard [16] (C_3). These three component codes are of slightly different rates, and length of C_3 is significantly different from the other two. To compensate for this difference in length, we use a different number of blocks in C_3 , so that the final length is very close to that of C_1 . Also, it should be noted that the coding scheme introduced in [4] with C_1 uses a single termination block ($T = 1$ in (2)).

Table I lists the encoding and decoding parameters of each coding scheme, along with the component code rate (R_{comp}) and staircase code rate (R_{st}). Component code decoder used for C_1 is the Chase-Pyndiah algorithm [9], and for LDPC-staircase codes, it’s the well-known SPA [12]. **Number of inner iterations of SPA** is set to 10 for C_2 , and 20 for C_3 , because of its longer component code. Additionally, for C_2 and C_3 , **number of outer iterations for first sliding window decoding operation ($v_{I,max}$)** is set to 24, and t_{flip} is set to 100 for the bit-flipping scheme. Fig. 2 presents BER performance of the three codes, along with Shannon bounds for the three rates. Performance of C_2 and C_3 without the bit-flipping scheme proposed in Section III are also included in the figure.

TABLE I. ENCODING AND DECODING PARAMETERS OF $\mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3$

	C_{comp}	R_{comp}	L	T	R_{st}	w	v_{max}	I_{max}
$\mathcal{C}_1$	(504,468)	0.9285	1000	1	0.8570	7	8	n/a
$\mathcal{C}_2$	(464,434)	0.9353	1000	2	0.8706	7	8	10
$\mathcal{C}_3$	(1440,1344)	0.9333	122	2	0.8656	7	8	20

Fig. 2 shows that the gain of SDD for BCH-staircase code is only around 0.3dB. Both LDPC-staircase codes, even without the bit-flipping modification, can be seen to outperform the soft decoded BCH-staircase code by significant margins. At a BER of 10^{-8} , the gap between SDD of $\mathcal{C}_1$ and $\mathcal{C}_2$ is about 0.55dB, while that between $\mathcal{C}_1$ and $\mathcal{C}_3$ is close to 1dB. The results suggest that LDPC component codes are a better choice for soft-decoded staircase codes. This is expected, since LDPC codes are much better suited for SDD [12] than the more algebraic BCH codes.

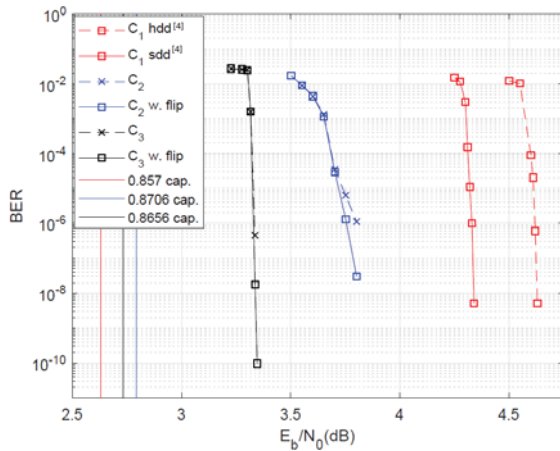


Fig. 2. Perf. comparison of soft-decoded BCH and LDPC staircase codes

For $\mathcal{C}_2$, bit-flipping scheme provides a substantial gain in performance. Indeed, performance without bit-flipping seems to be tending towards an error-floor. For $\mathcal{C}_3$, performance gain from bit-flipping does not seem that significant, but a closer look, given in Fig. 3, shows that there also, BER performance is improved from around 10^{-6} to 10^{-8} at a E_b/N_0 value of 3.3dB. Due to the characteristics of the component code of $\mathcal{C}_3$, particularly its longer length and higher girth, performance without bit-flipping does not show signs of an error-floor.

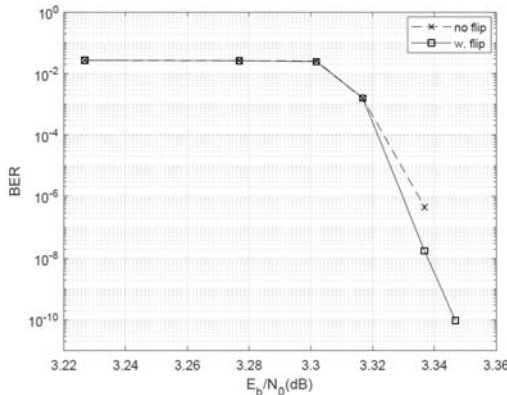
Fig. 3. Perf. improvement of $\mathcal{C}_3$ with bit-flipping

Fig. 2 also shows a 0.45dB gap between curves of $\mathcal{C}_2$ and $\mathcal{C}_3$, again at a BER of 10^{-8} . Furthermore, the curve of $\mathcal{C}_2$ has a more gentle slope, compared to the very steep curves of $\mathcal{C}_1$ and $\mathcal{C}_3$. Reasons for both the gap and the slope are the characteristics of the component code used for $\mathcal{C}_2$. Being a short LDPC code (length 464) of very high rate (0.935), its parity-check matrix has to have a number of lower degree columns for a reasonably good girth. In fact, the code was constructed from an ‘ultra sparse’ graph of regular column weight 2 [15]. It is noted in [17] that degree 2 variable nodes generally converge slower and has a detrimental effect on the slope of BER curve. Still, for a short length LDPC code of such a high rate, that seems to be the only viable option.

With the three codes being of slightly different rates, their performance with respect to the specific Shannon bound is also interesting. $\mathcal{C}_1$ shows a large gap of around 2dB with HDD, and 1.7dB with SDD, and for $\mathcal{C}_2$, its about 1dB, all at a BER of 10^{-8} . As expected, $\mathcal{C}_3$, which has a longer component code, has the smallest gap with capacity, which is around 0.6dB at a BER of 10^{-10} .

Since the same window size (7 blocks) is used for all 3 schemes, latency-wise, $\mathcal{C}_3$ is at a disadvantage when compared with others. $\mathcal{C}_2$ has the lowest latency, due to its component code being slightly shorter than that of $\mathcal{C}_1$. Component decoder of $\mathcal{C}_1$, Chase-Pyndiah algorithm, includes sorting of input bit reliabilities, a number of HDD operations, and calculation of Euclidean distances [9]. Since the code is a very high rate BCH code, with a lower minimum distance, number of ‘test patterns’ in Chase-Pyndiah decoding would be relatively low. This results in the overall complexity being of a tolerable level. SPA, component decoder of $\mathcal{C}_2$ and $\mathcal{C}_3$, can also be considered to be of a similar complexity level, especially when implemented with early termination, and a relatively low number of maximum iterations [12]. Thus, it is fair to conclude that decoding complexities of $\mathcal{C}_1$ and $\mathcal{C}_2$ are of similar levels, as the component codes are of similar lengths, and their decoders are of similar complexity. Due to the higher length of the component code of $\mathcal{C}_3$, which requires a higher number of SPA iterations to successfully decode, its decoding complexity is relatively higher. With each staircase code comprised of a large number of blocks, using a larger number of outer iterations at the start for $\mathcal{C}_2$ and $\mathcal{C}_3$ is not expected to have much impact on complexity. Bit-flipping scheme does increase decoding complexity, but when implemented using ‘syndrome updates’ as presented in section III, and with t_{flip} kept at a moderate value, as was done here, this increase is marginal.

Fig.4 presents the BER performance of LDPC-staircase codes of moderately high rate, both with and without the bit-flipping scheme. We consider two coding schemes, one with a (672, 560) component code ($\mathcal{C}_4$), and the other with a (960, 800) component code ($\mathcal{C}_5$). Both are of rate $\frac{5}{6}$, and are specified in IEEE 802.16 standard [18]. We use 500 blocks in $\mathcal{C}_4$ and 250 blocks in $\mathcal{C}_5$, and final rates are 0.66607 and 0.6655, respectively. Decoding parameters were set as $w = 8$, $v_{max} = 10$, $v_{I,max} = 30$, $I_{max} = 20$ and $t_{flip} = 100$ for both codes.

It can be seen that both $\mathcal{C}_4$ and $\mathcal{C}_5$ have quite steep BER curves, even without the bit-flipping scheme. Due to their rates being almost exact, respective Shannon bounds are also very close. While $\mathcal{C}_4$, with the shorter component code, has a gap of

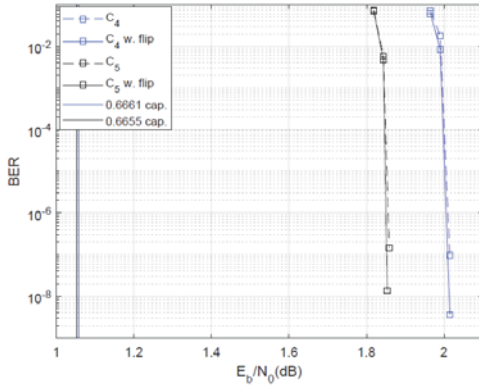


Fig. 4. Performance of moderate rate LDPC-staircase codes

a bit less than 1dB with capacity, that decreases to about 0.8dB with C_5 , due to its larger component code length. This gap can be expected to reduce even further if a longer component code is used.

Bit-flipping scheme seems to offer considerable performance advantages to both C_4 and C_5 . In the case of C_4 , BER improves from around 10^{-7} to 3×10^{-9} at 2.01dB, while for C_5 , improvement is from 10^{-7} to 10^{-8} at 1.85dB. These gains are even more significant since the complexity increase, comparatively, is much smaller. As explained in Section III, additional calculations required for bit-flipping are mostly XOR operations, and even these are kept to a minimum through the parameter t_{flip} , which is set to 100 for the simulations.

For both sets of codes simulated, scaling factors for updating extrinsic information were set to $\alpha_c = 0.75$ and $\alpha_n = 0.375$. It should be noted that these coefficients were not optimized, and marginal improvements of performance might be possible with using optimized scaling factors.

One weakness of these codes observed during simulations was error propagation. Since sliding window decoding uses the last output block for current decoding operation, any errors present in that block may force the next output block to be erroneous as well. Straightforward method for addressing this is to use scaling factors to reduce the magnitude of possible erroneous estimates, as was done here with the proposed scheme (in Section II-C). Better options of addressing error-propagation, such as using *damped* forms¹ of BP [19], can be expected to improve performances of the scheme significantly.

V. CONCLUSION

This paper proposes using LDPC codes as component codes of soft-decoded staircase codes. Certain modifications are suggested for the traditional sliding window decoding scheme, which help improve performance of LDPC-staircase codes. Low complexity bit-flipping scheme proposed is particularly effective in lowering the error-floors. Latency of sliding window decoding is minimal, and its hardware implementations can be easily parallelized due to the special structure of staircase codes. Results presented show that LDPC-staircase codes outperform BCH-staircase codes with SDD in high rate

regime, and suggest that they are also good candidates for applications which do not require very high rates. Decoding scheme suggested here may be improved further, leading to even better performance.

LDPC-staircase codes share a lot of similarities with spatially-coupled LDPC (SC-LDPC) codes [13], which were briefly touched upon in Section II. Unique advantages of LDPC-staircase codes, particularly in encoding and decoding complexities, and disadvantages, in comparison to traditional SC-LDPC codes is a topic that warrants further discussion.

REFERENCES

- [1] B. P. Smith, A. Farhood, A. Hunt, F. R. Kschischang, and J. Lodge, "Staircase Codes: FEC for 100 Gb/s OTN", *Journal of Lightwave Technology*, vol. 30, no. 1, pp. 110-117, Jan. 2012
- [2] L. Holzbaur, H. Bartz, and A. Wachter-Zeh, "Improved decoding and error floor analysis of staircase codes", *Designs, Codes, and Cryptography*, vol. 87, no. 2-3, pp. 647-664, Mar. 2019
- [3] M. Qiu, L. Yang, Y. Xie, and J. Yuan, "Terminated staircase codes for NAND flash memories", *IEEE Transactions on Communications*, vol. 66, no. 12, pp. 5861-5875, Dec. 2018
- [4] X. Dou, M. Zhu, J. Zhang, and B. Bai, "Soft-decision based sliding-window decoding of staircase codes", *Proceedings of IEEE Intern. Symp. on Turbo Codes and Iterative Inf. Proc.*, Hong Kong, China, Dec. 2018
- [5] G. Hu, and J. Sha, "Tail-biting code: A modification to staircase code for high-speed networking", *Proceedings of IEEE Workshop on Signal Processing Systems*, Hangzhou, China, Oct. 2015
- [6] M. Barakatain, and F. R. Kschischang, "Low-complexity concatenated LDPC-staircase codes", *Journal of Lightwave Technology*, vol. 36, no. 12, pp. 2443-2449, June 2018
- [7] L. M. Zhang, and F. R. Kschischang, "Complexity-optimized concatenated LDGM-staircase codes", *Proceedings of IEEE International Symposium on Information Theory*, Aachen, Germany, June 2017
- [8] Y. Lei, A. Alvarado, B. Chen, X. Deng, Z. Cao, J. Li, and K. Xu, "Decoding staircase codes with marked bits", *Proc. of IEEE Intern. Symp. on Turbo Codes and Iterative Inf. Proc.*, Hong Kong, China, Dec. 2018
- [9] R. M. Pyndiah, "Near-optimum decoding of product codes: Block turbo codes", *IEEE Tran. on Comm.*, vol. 46, no. 8, pp. 1003-1010, Aug. 1998
- [10] L. Zhou, B. Feng, J. Jiao, K. Liang, *et al*, "Performance analysis of soft decoding algorithms for polar-staircase coding scheme", *Proc. of Int. Conf. on Wireless Comm. and Sig. Proc.*, Hangzhou, Ch., Oct. 2018
- [11] R. G. Gallager, "Low-density parity-check codes", *IRE Transactions on Information Theory*, vol. IT-8, pp. 21-28, Jan. 1962
- [12] D. J. C. MacKay, "Good error-correcting codes based on very sparse matrices", *IEEE Tran. on Inf. Th.*, vol. 46, no. 2, pp. 399-431, Mar. 1999
- [13] D. G. M. Mitchell, M. Lentmaier, and D. J. Costello, Jr., "Spatially coupled LDPC codes constructed from protographs", *IEEE Transactions on Information Theory*, vol. 61, no. 9, pp. 4866-4889, Sep. 2015
- [14] G. Lechner, and J. Sayir, "On the convergence of log-likelihood values in iterative decoding", *Mini-Workshop on Topics in Inf. Th.*, Sept. 2002.
- [15] D. Declercq, "Regular ultra-sparse graphs and related non-binary LDPC codes", [Online]. Available: <https://perso-etis.ensea.fr/~declercq/graphs.php> [Accessed: 20- Mar- 2019]
- [16] IEEE Std 802.15.3c-2009 (Amendment to IEEE Std 802.15.3-2003), "Part 15.3: Wireless MAC and PHY spec. for high rate WPANs, Amendment 2: Mm-wave-based alt. phy. layer ext.", pp. 79-81, Oct. 2009
- [17] W. Yang, W. E. Ryan, and Y. Li, "Design of efficiently encodable moderate-length high-rate irregular LDPC codes", *IEEE Transactions on Communications*, vol. 52, no. 4, pp. 564-571, Apr. 2004
- [18] IEEE Std 802.16-2009 (Revision of IEEE Std 802.16-2004), "Part 16: Air interface for broadband wireless acc. sys.", pp. 1135-1138, May 2009
- [19] L. Cohen, and R. Zivan, "Max-sum revisited: The real power of damping", *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, Sao Paulo, Brazil, May 2017

¹combining current and previous estimations using a damping factor